**Angry Bullet Game - Assignment 2 Report**

**Author:** SEDA GUNES
**Date:** November 17, 2025
**AI Assistants Used:** OpenAI's ChatGPT (GPT-5), Google Gemini, Claude

---

## 1. Project Overview

The Angry Bullet game is a physics-based projectile motion simulation where players aim and fire a bullet to hit an orange target while avoiding gray obstacles. The game demonstrates object-oriented programming principles, physics simulation, and interactive graphics using Java and the StdDraw library.

**Game Objective:** Launch a bullet from a fixed platform, adjust velocity and angle using mouse position, hit the target while avoiding obstacles, and retry upon failure or success.

---

## 2. System Architecture

### 2.1 Class Structure

The application consists of six classes organized in an object-oriented hierarchy:

**Main.java** - Entry point that instantiates GamePlay and starts the game loop.

**GamePlay.java** - Core controller managing game state, user input, rendering, and collision detection. Key responsibilities include loading configuration from file, handling mouse/keyboard input, controlling the animation loop, drawing trajectory paths, and managing game states (aiming, shooting, awaiting restart).

**Bullet.java** - Represents the projectile with position tracking and physics calculations. Implements projectile motion equations: $x(t) = x_o + v_x \times t$ and $y(t) = y_o + v_y_o \times t - 0.5 \times g \times t^2$ with gravity constant $g = 9.81$ m/s². Updates position each frame and renders itself as a filled circle.

**Shape.java** - Abstract base class providing common fields (position, dimensions) and interface methods (draw(), isColliding()) for all game objects. Uses the template method pattern with protected fields accessible to subclasses.

**Obstacle.java** - Rectangular barriers extending Shape. Renders as dark gray or black filled rectangles and implements circle-rectangle collision detection using the Circle vs. AABB algorithm.

**Target.java** - Goal object extending Shape. Renders as an orange filled rectangle and uses the same collision detection algorithm as Obstacle.

---

## 3. Key Features and Implementation

### 3.1 Physics Simulation

The game implements realistic projectile motion with velocity decomposition into horizontal and vertical components. Gravity provides constant downward acceleration, creating a parabolic trajectory. Position updates occur every 0.08 seconds using time-step animation for smooth motion.

### 3.2 User Interaction

**Mouse-Based Aiming:** Distance from the shooting point determines velocity (scaled by 0.8, max 300 units), while angle is calculated using Math.atan2() and constrained to 0-90 degrees. A dynamic aiming line scales with velocity, and real-time angle/velocity values are displayed.

**Keyboard Controls:** SPACE to fire, R to restart after outcome.

**Visual Feedback:** Dotted trajectory prediction line updates in real-time during aiming and shows the complete path after the bullet lands.

### 3.3 Collision Detection

Uses the Circle vs. AABB (Axis-Aligned Bounding Box) algorithm: finds the closest point on the rectangle edge to the bullet center, calculates distance, and collision occurs if distance is less than bullet radius. This applies to target detection (success), obstacle detection (failure), and ground collision.

### 3.4 Game State Management

Three states control game flow:

1. **Aiming Mode** - User adjusts angle and velocity with mouse
2. **Bullet Moving** - Physics simulation in progress with collision checking
3. **Awaiting Restart** - Outcome displayed, waiting for 'R' key

A critical implementation detail: shotV0 and shotTheta values are saved at the moment of firing to ensure the trajectory remains fixed even if the mouse moves during bullet flight.

---

## 4. Object-Oriented Design Principles

**Encapsulation:** All fields are private with public getter methods, hiding internal state and providing clear public interfaces.

**Inheritance:** Shape serves as the base class for Obstacle and Target, enabling code reuse through shared fields and structure while allowing specialized behavior.

**Polymorphism:** The draw() and isColliding() methods are overridden in subclasses, allowing lists of Shape objects to contain mixed types with runtime method dispatch.

**Composition:** GamePlay contains a Bullet object and aggregates lists of Shape objects, demonstrating "has-a" relationships.

---

**5. Configuration and Extensibility**

Game objects are loaded from an external config.txt file with format TYPE,x,y,width,height where TYPE is either OBS (obstacle) or TGT (target). This allows easy level design without code changes.

Key tunable constants include: INITIAL_V0 = 180.0, INITIAL_THETA = 45.0, MAX_V0 = 300.0, screen dimensions (700×400), and bullet radius (4.0).

---

**6. Technical Challenges and Solutions**

**Fixed Trajectory Drawing:** To prevent trajectory changes during flight, velocity and angle values are captured at firing time and used for all subsequent trajectory calculations, separating aiming parameters from shot parameters.

**Smooth Animation:** Double buffering (StdDraw.enableDoubleBuffering()), fixed 0.08-second time steps, and consistent 20ms frame pauses create smooth motion.

**Accurate Collision Detection:** Circle vs. AABB algorithm provides precise collision detection by finding the closest rectangle point to the bullet center, avoiding false positives from simple bounding box checks.

**Trajectory Visibility:** Dotted lines using small circles at intervals (every 3rd time step) create a visible but unobtrusive trajectory path.

---

**7. Game Flow**

The game follows a cycle: Initialize StdDraw → Load config file → Enter aiming mode (track mouse, display aiming line) → Fire on SPACE press → Bullet moving (update position, check collisions, draw trajectory) → Display outcome → Wait for 'R' to restart → Return to aiming mode.

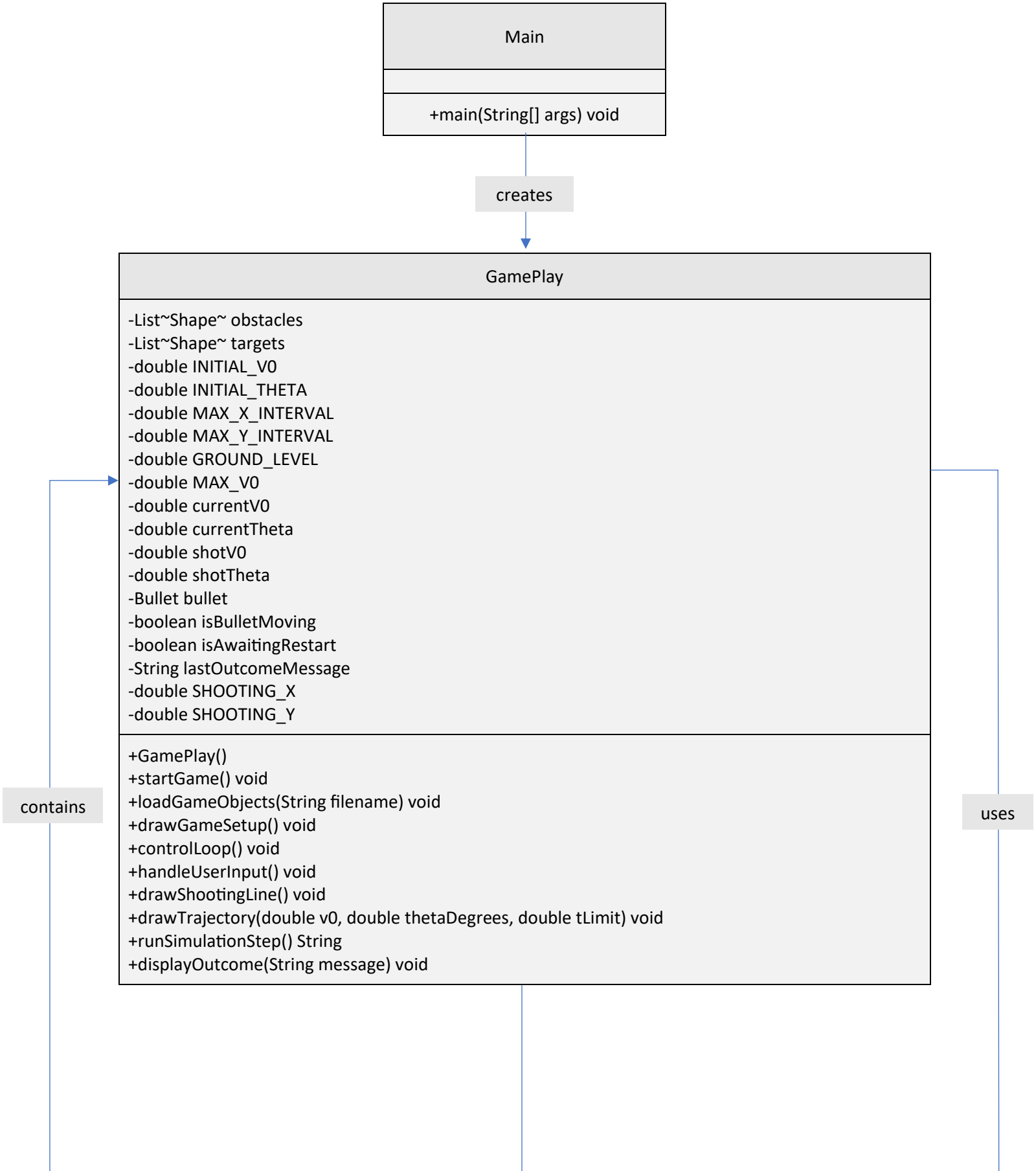
---

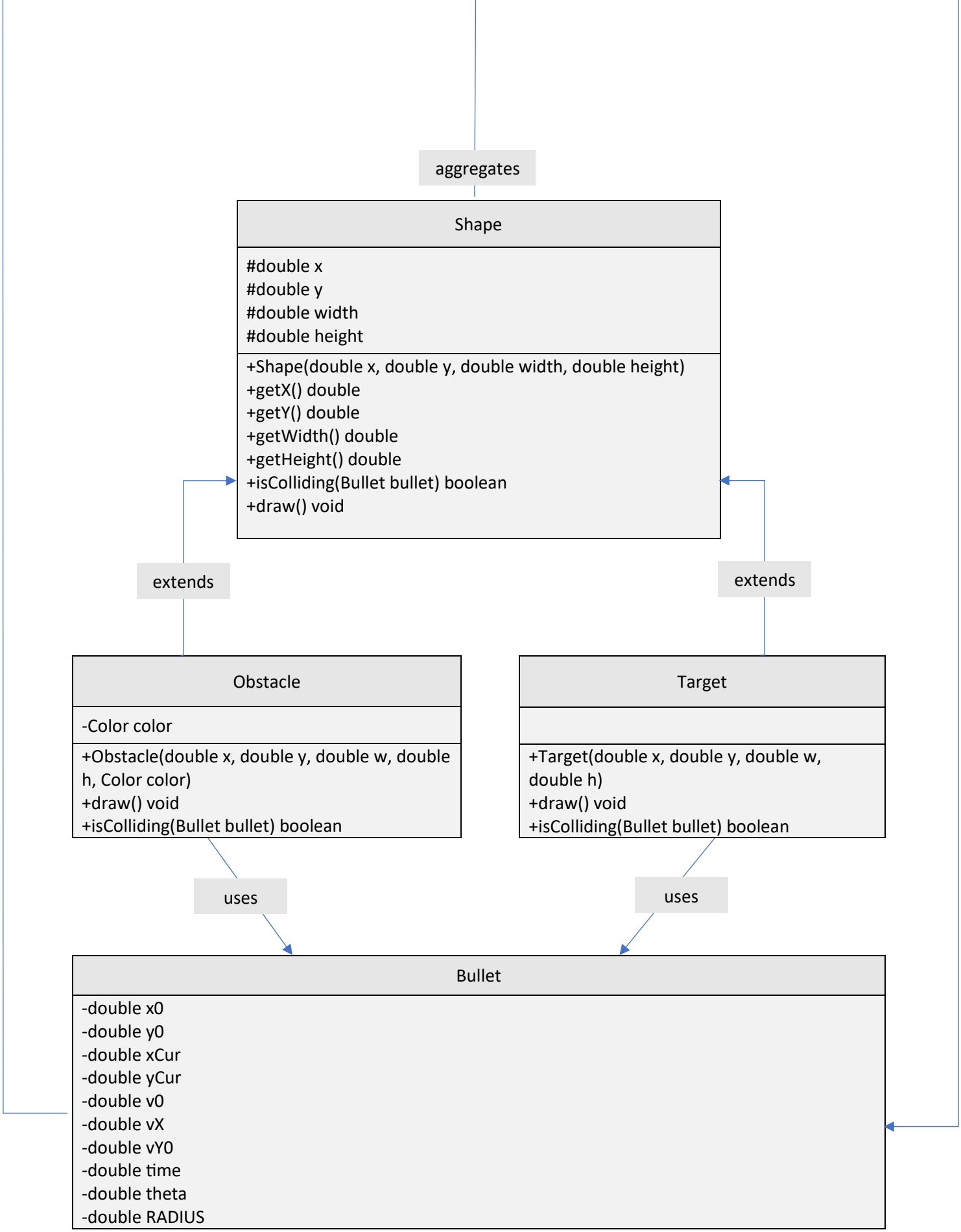
**8. Conclusion**

The Angry Bullet game successfully demonstrates fundamental concepts of object-oriented programming, physics simulation, interactive graphics, and game state management in Java. The modular design with clear separation of concerns allows for easy extension and modification. The external configuration system enables flexible level design, while the implementation of proper OOP principles (encapsulation, inheritance, polymorphism)

---

**9. UML class diagrams**




aggregates
Shape
#double x
#double y
#double width
#double height
+Shape(double x, double y, double width, double height)
+getX() double
+getY() double
+getWidth() double
+getHeight() double
+isColliding(Bullet bullet) boolean
+draw() void
extends
extends
Obstacle
-Color color
+Obstacle(double x, double y, double w, double h, Color color)
+draw() void
+isColliding(Bullet bullet) boolean
Target
+Target(double x, double y, double w, double h)
+draw() void
+isColliding(Bullet bullet) boolean
uses
uses
Bullet
-double x0
-double y0
-double xCur
-double yCur
-double v0
-double vX
-double vY0
-double time
-double theta
-double RADIUS

```
+double g$
+Bullet(double x0, double y0, double v0, double theta)
+getX0() double
+getY0() double
+getxCur() double
+getyCur() double
+getV0() double
+getTheta() double
+getRadius() double
+getvX() double
+getvY0() double
+getRADIUS() double
+getTime() double
+updatePosition(double deltaTime) void
+draw() void
```

### 10. Sample Gameplay Screens

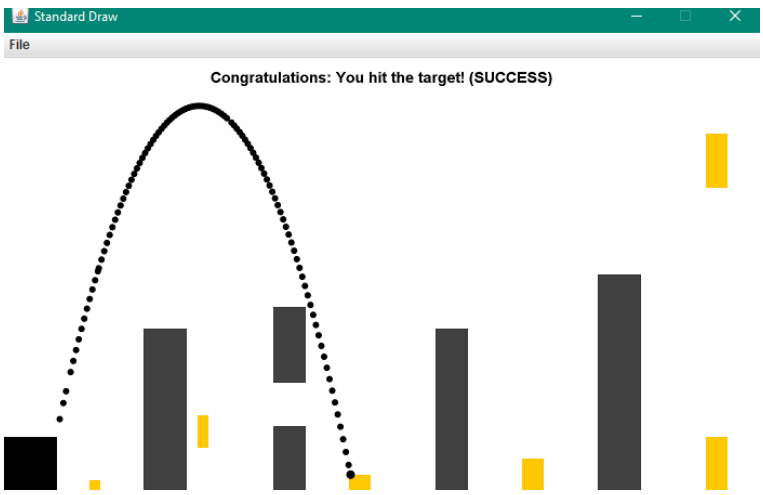



Figure 1. Case for hitting the target (orange box). Successful shot hits one of the orange targets.

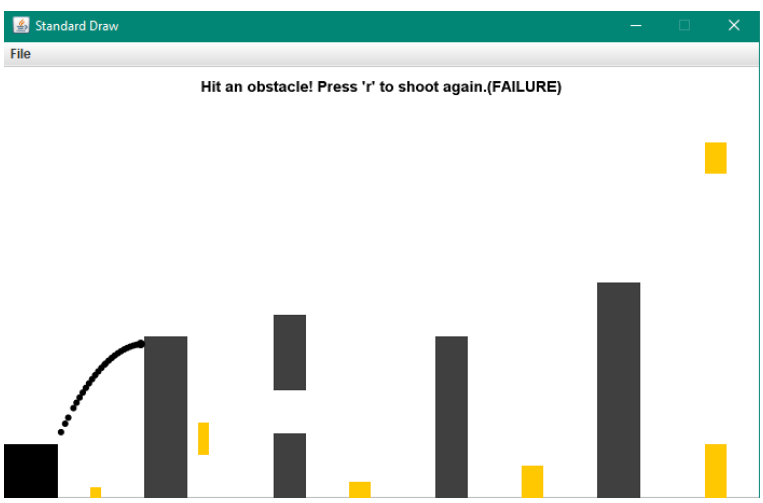


Figure 2. Case for hitting an obstacle.

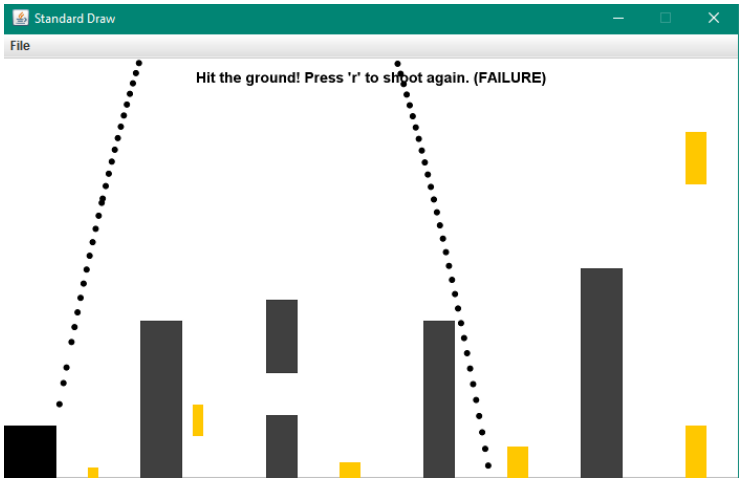



Figure 3. Case for ground contact. Shooting simulation should finish in this case.

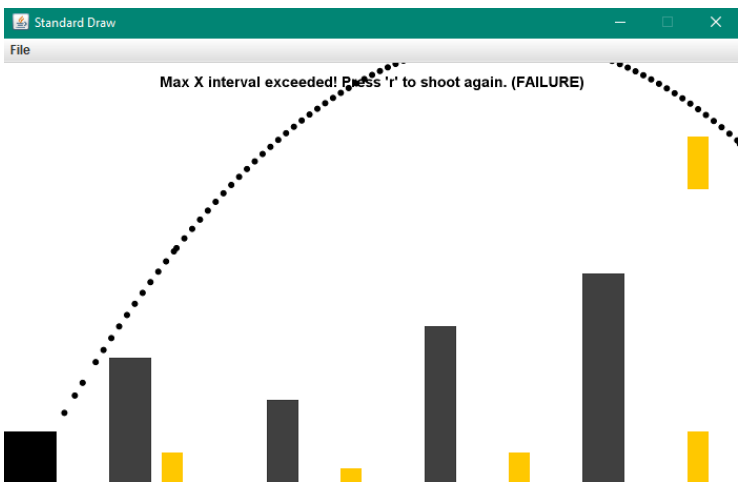



Figure 4. Case for Max X coordinate reach. Shooting simulation should finish in this case

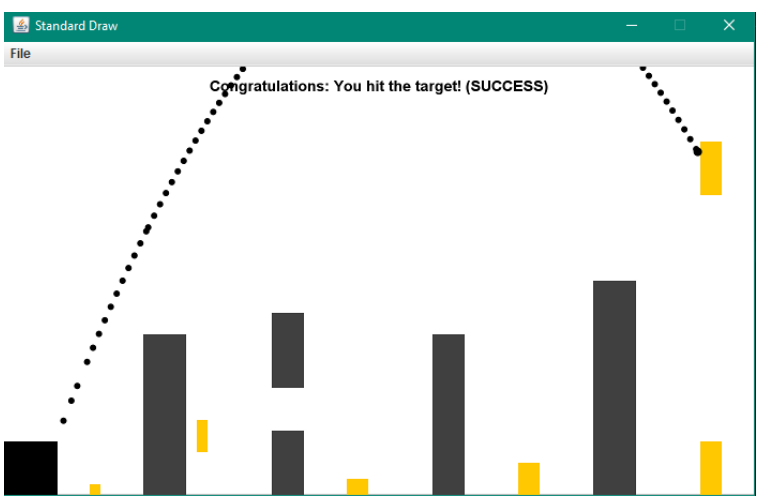



Figure 5. Bullet is allowed to travel through the upper part of the sky.